Reproducing DoRA on Mistral-7B-Instruct-v0.3: Task accuracy, format adaptation, and the rank trade-off

Remilia Scarlet
Student number 1919810, remilia.scarlet@student.unimelb.edu.au

Abstract

Parameter-efficient fine-tuning (PEFT) adapts large language models by training a tiny fraction of their weights. Low-Rank Adaptation (LoRA) is the standard choice, but Weight-Decomposed Low-Rank Adaptation (DoRA; Liu et al., 2024) reports higher accuracy — with its largest gains at low rank — by splitting each weight into a magnitude and a direction and applying the low-rank update only to the direction. We test this claim as a generalization study: instead of the paper's raw LLaMA base, we adapt the instruction-tuned `Mistral-7B-Instruct-v0.3` on the per-task BoolQ benchmark and the multi-task commonsense-170k mixture, sweeping rank $r \in \{4, 8, 16\}$ against a matched LoRA baseline at a course-feasible 2,500-step budget (three seeds) and a 10,000-step extension (four fresh seeds).

Across 60 training runs on University of Melbourne Spartan H100 GPUs, DoRA does **not** improve task accuracy over LoRA at any rank. At mid rank (r=8) it instead better resists the format shift that multi-task training induces, retaining the original yes/no answer format where LoRA adopts the trained true/false (a +0.16 edge on the yes/no likelihood probe, outside seed noise). Quadrupling the budget sharpens an *inverse-rank* trend: low-rank adapters stay at or above the zero-shot baseline (generation exact-match, "genmatch", 0.82) while high-rank adapters drift well below it (the within-method gap widening from $\sim$ 3–7 to $\sim$ 14–16 percentage points). A parser fix separating *task accuracy* from *format adaptation* proved essential, and DoRA's $\sim 3.3\times$ wall-time and $\sim 1.8\times$ peak-memory overhead is reconfirmed throughout. We conclude that DoRA's low-rank advantage surfaces here on a format-related axis rather than as accuracy — a conclusion tempered by the limited seed count, the sub-paper training budget, and the already-aligned instruction-tuned base.

1 Introduction

Fine-tuning a 7-billion-parameter model the conventional way means updating every weight and storing optimizer state for each — tens of gigabytes of GPU memory and a fresh multi-gigabyte checkpoint per task. Parameter-efficient fine-tuning (PEFT) removes that barrier: LoRA freezes the pretrained weights and learns a small low-rank update BA in their place, training under 0.1% of the parameters and shipping a $\sim$ 30 MiB adapter instead of a 14 GiB model. This makes per-task specialization cheap enough to run on a single GPU — but LoRA leaves a measurable accuracy gap below full fine-tuning, and that gap is widest exactly where we most want to save: at low rank. DoRA targets this gap directly. It decomposes each pretrained weight into a magnitude and a direction, applies the LoRA-style low-rank update only to the direction, and learns the magnitude as a separate parameter; the paper reports that this recovers much of the lost accuracy and that the advantage is *largest at low rank*, where parameter budget is tightest.

If that claim is robust, it is genuinely useful: it would mean cheaper, smaller adapters with less quality loss, directly improving how well large models can be specialized on constrained hardware. But a single paper's numbers, measured on one base model and one training budget, do not by themselves establish robustness — and the extra cost DoRA incurs ($\sim 3.3\times$ training time in our runs) only pays off if its accuracy advantage actually transfers. This project therefore tests the paper's central claim under deliberately different conditions: an instruction-tuned base model (`Mistral-7B-Instruct-v0.3`) rather than the paper's raw LLaMA, the same datasets the paper uses (BoolQ and commonsense-170k) but at a realistic course compute budget, and a controlled LoRA-versus-DoRA rank sweep with multiple seeds so we can tell a real effect from seed noise. The question we carry throughout is the paper's own: *does DoRA outperform LoRA more at low rank?* — and, where it does not, *what does the magnitude–direction decomposition change instead?*

2 Problem description

Objective and success criterion. The goal is to test, not merely re-run, DoRA's headline result. Because we change the base model, the PEFT implementation (HuggingFace `peft` rather than the authors' custom fork), and the training budget, bit-identical numbers are neither expected nor the point. Success is answering a qualitative question with controlled evidence: *across a matched LoRA/DoRA rank sweep, does DoRA's reported low-rank accuracy advantage appear in our regime, and if not, what — if anything — does the decomposition measurably change?* Framing the study as a generalization test rather than a strict replication is what makes a null accuracy result informative rather than a failed reproduction.

Model and data. The base model is `Mistral-7B-Instruct-v0.3` (Jiang et al., 2023), a 7.25 B-parameter instruction-tuned model that is open-access and already part of the course toolchain. We adapt it in two regimes:

- **BoolQ** (Clark et al., 2019) — a binary yes/no reading-comprehension QA dataset, 9,427 training and 3,270 development examples. This is the *per-task* regime: the adapter sees only BoolQ.
- **commonsense-170k** (cs170k) — a 170,000-example mixture of eight commonsense-reasoning tasks (BoolQ among them), assembled by the LLM-Adapters project (Z. Hu et al., 2023) and used by the DoRA paper. This is the *multi-task* regime, and the one the paper's low-rank claim is built on.

Both regimes are evaluated on the *same* full BoolQ development set (3,270 examples), so a single accuracy axis is comparable across them and against a zero-shot (no-adapter) baseline.

Difficulties and obstacles. Several constraints shaped the design and the interpretation:

1. **Compute and memory.** DoRA's decomposition roughly triples training wall-time and raises peak GPU memory to $\sim$ 60 GB (versus $\sim$ 34 GB for LoRA), so it will not fit on a 24 GB consumer card; all training ran on data-centre H100 80 GB GPUs (the Methods quantify the resulting GPU-hour cost).
2. **Training budget versus the paper.** The paper trains $\sim$ 32k cs170k steps; a 2-method $\times$ 3-rank $\times$ 3-seed grid cannot match that on a course budget. We cap the core study at 2,500 steps (7.8% of the paper's — training loss converges by step $\sim$ 1,200) and add a 10,000-step extension (31%) to probe how the conclusions move with budget.
3. **Statistical power.** Three seeds (core) and four (extension) are too few for formal significance, and seed variance is large on some metrics. We report mean $\pm$ std throughout and state explicitly when a gap sits within noise.
4. **Base-model mismatch (the key interpretive obstacle).** An instruction-tuned model is already format-stable and competent at BoolQ-style prompts, which suppresses precisely the format-collapse failure mode that the paper's low-rank DoRA advantage is most sensitive to. A null accuracy result therefore has to be read in light of a starting point that gives LoRA less room to fail.
5. **Metric ambiguity.** The BoolQ evaluation prompt asks for *yes/no*, but cs170k teaches the model to answer *true/false*. A naive exact-match parser that accepts only true/false scores a correct yes/no answer as wrong — silently conflating *task accuracy* (is the answer right?) with *format adaptation* (did the adapter adopt the training vocabulary?). Recognizing this and broadening the parser to accept either vocabulary was necessary to interpret the cs170k results at all, and the task-versus-format distinction becomes a central thread in the Methods and Results.

3 Techniques and methods

3.1 From LoRA to DoRA

Both methods adapt a frozen pretrained weight $W_0 \in \mathbb{R}^{d\times k}$ with a small trainable update. **LoRA** (E. J. Hu et al., 2021, developed in full in the Foundations section, Part 5) writes the adapted weight as

$$W = W_0 + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k), \tag{1}$$

freezing W_0 and training only the rank-r factors A and B; the scalar α/r keeps the update's scale stable as r varies. **DoRA** (Liu et al., 2024) keeps this low-rank update but first splits each weight into a *magnitude* and a *direction*.

Writing $\|\cdot\|_c$ for the column-wise Euclidean norm, it reparameterizes the adapted weight as

$$W' = m \frac{W_0 + BA}{\|W_0 + BA\|_c}, \tag{2}$$

where $m \in \mathbb{R}^{1\times k}$ is a trainable magnitude vector (one entry per column) and $(W_0 + BA)$ is the LoRA-updated direction, renormalized to unit column-norm before m rescales it. At initialization $m = \|W_0\|_c$ and $BA = 0$, so $W' = W_0$ exactly — training starts from the pretrained model, as in LoRA. The motivation is that full fine-tuning tends to move a weight's magnitude and direction by different relative amounts, whereas LoRA's single additive term couples them; giving the magnitude its own parameter lets DoRA adjust scale and orientation independently, which the paper argues better mimics full fine-tuning and is most beneficial when the directional budget — the rank r — is small.

For a fair comparison the two methods share an otherwise identical configuration: we use HuggingFace `peft`'s `LoraConfig` (Mangrulkar et al., 2022), and the *only* difference between a LoRA run and its DoRA counterpart is the `use_dora` flag. We deliberately use the library implementation rather than the authors' bundled fork — the question is whether a practitioner reaching for the standard tool sees the paper's advantage.

3.2 Experimental design

The study is a controlled factorial sweep: 2 methods (LoRA, DoRA) × 3 ranks ($r \in \{4, 8, 16\}$) × multiple seeds × 2 training regimes, plus a zero-shot (no-adapter) baseline. The **core study** (Tier 2 in the figures) is 36 runs over three seeds $\{42, 1, 2\}$, spanning the per-task BoolQ regime and the multi-task cs170k regime. A sequential **budget extension** (Tier 3) adds 24 cs170k-only runs over four *fresh* seeds $\{114, 514, 1919, 810\}$ at four times the training length; the disjoint seed set makes it an independent replication rather than a re-aggregation of the core runs.

Across every run the adapter targets *all* linear layers (`target_modules: all-linear` — both the attention $q/k/v/o$ projections and the MLP gate/up/down projections), with $\alpha/r = 2$ (so $\alpha = 8, 16, 32$ for $r = 4, 8, 16$), LoRA dropout 0.05, and bf16 weights. Training uses the HuggingFace `Trainer` (Wolf et al., 2020) with its default AdamW optimizer and a linear warmup-then-decay learning-rate schedule, an effective batch size of 16 (micro-batch 4 × gradient accumulation 4), and a 512-token cap. The per-regime settings are:

Table 1: *Per-regime training settings.*

Regime (tier)	Train set	Steps	Warmup	Learning rate
BoolQ (core)	BoolQ train (9,427)	500	50	5×10^{-5}
cs170k (core)	commonsense-170k	2,500	100	2×10^{-4}
cs170k (extension)	commonsense-170k	10,000	400	2×10^{-4}

The multi-task regime uses a learning rate four times higher than the per-task one, reflecting its larger, more heterogeneous mixture; it is also the regime in which the format-adaptation effects discussed in the Results arise, since cs170k trains the model toward a *true/false* answer format.

3.3 Evaluation, and a necessary parser fix

Every adapter — whatever its training regime — is evaluated on the *same* full BoolQ development set (3,270 examples), giving one accuracy axis comparable across regimes and against the zero-shot baseline. We report two metrics:

- **Likelihood accuracy** compares the first-token log-probabilities the model assigns to "Yes" versus "No". It needs no text parsing and probes directly whether the model's internal distribution favours the correct answer.
- **Generation exact-match (genmatch)** greedily decodes a short answer, parses it, and exact-matches it to the gold label — the paper-style, user-facing metric.

The genmatch metric exposed a subtlety that became central to the analysis. The BoolQ evaluation prompt asks for *yes/no*, but the cs170k training data teaches *true/false*. A strict parser that accepts only true/false therefore scores a model answering "yes" correctly as *wrong*, conflating two different things: whether the answer is right (*task accuracy*) and whether the adapter has adopted the trained vocabulary (*format adaptation*). We resolve this

by reporting genmatch under two parsers — a **broad** parser (`parse_yes_no_or_true_false`) that accepts either vocabulary normalized to the gold label, measuring task accuracy; and a **strict** parser (true/false only), which instead measures *how strongly* the adapter has overridden the prompt with the trained format. This task-versus-format split is the lens for the Results, and it becomes essential at the longer extension budget, where the model commits so fully to true/false that the yes/no likelihood probe collapses even while the model keeps answering correctly.

3.4 Compute environment

Training ran on the University of Melbourne Spartan HPC `gpu-h100` (80 GB) partition (`torch 2.6.0+cu124`). DoRA's decomposition costs roughly 3.3× the wall-time of LoRA at matched settings and peaks near 60 GB of GPU memory (versus ∼ 34 GB for LoRA) at every rank, so DoRA training requires 80 GB-class hardware. The core sweep totals ∼ 25 GPU-hours of training plus generation-eval; the budget extension adds ∼ 114 GPU-hours. Each run records its full configuration and code commit alongside its metrics, which are aggregated into the summary tables reported next.

4 Presentation and discussion of results

All adapters are scored on BoolQ-dev accuracy against the zero-shot baseline (likelihood 0.824, broad-genmatch 0.820). Headline DoRA−LoRA gaps are quoted inline; the full per-cell tables (12 core cells, 6 extension cells, and the DoRA−LoRA gap tables) are collected in the Appendix.

4.1 Per-task BoolQ: both methods saturate, neither pulls ahead

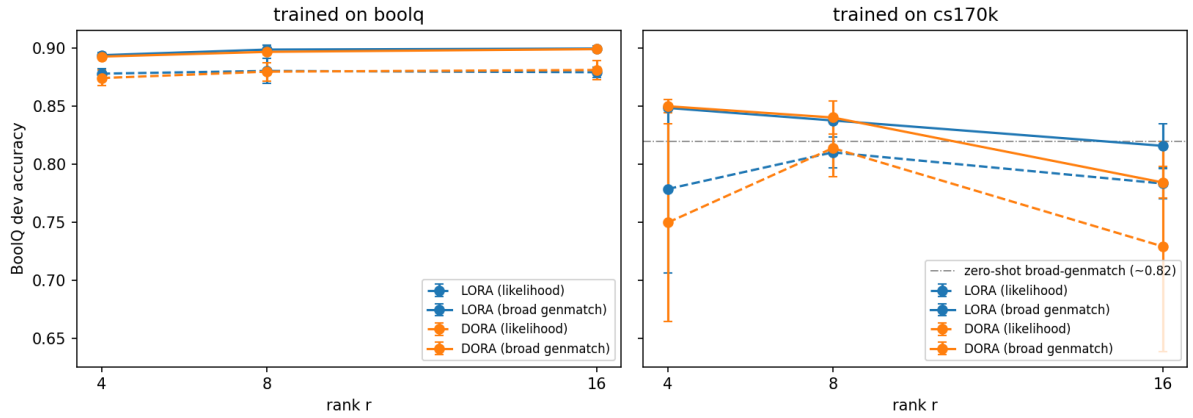

Figure 1: *BoolQ-dev accuracy versus rank for LoRA (blue) and DoRA (orange), solid = broad genmatch, dashed = likelihood. Left: trained on BoolQ. Right: trained on cs170k, with the zero-shot broad-genmatch baseline (≈ 0.82) marked.*

On BoolQ, both methods lift the model from the 0.82 baseline to ∼ 0.88 likelihood and ∼ 0.89–0.90 genmatch, essentially flat across rank (Figure 1, left). The two are indistinguishable: the DoRA−LoRA genmatch gap is ≤ 0.002 at every rank, well inside seed variation. The per-task regime is easy enough that a rank-4 adapter already saturates it, leaving no headroom for DoRA's decomposition to exploit — a first null for the paper's low-rank-advantage claim.

The loss curves (Figure 2) show both regimes converge quickly in training, but the cs170k eval loss — measured on the BoolQ yes/no prompt — *rises* toward ∼ 10 while its training loss collapses. A cs170k-trained model is answering well but in the "wrong" vocabulary for the yes/no eval, which is exactly the effect the next section makes precise.

4.2 Multi-task cs170k: an inverse-rank trend, and two views of "accuracy"

Training on the 8-task cs170k mixture is more revealing (Figure 1, right). On task accuracy (broad genmatch), a low-rank adapter *beats* the zero-shot baseline (r=4: ∼ 0.85), while a high-rank one falls *below* it (r=16: 0.82 LoRA

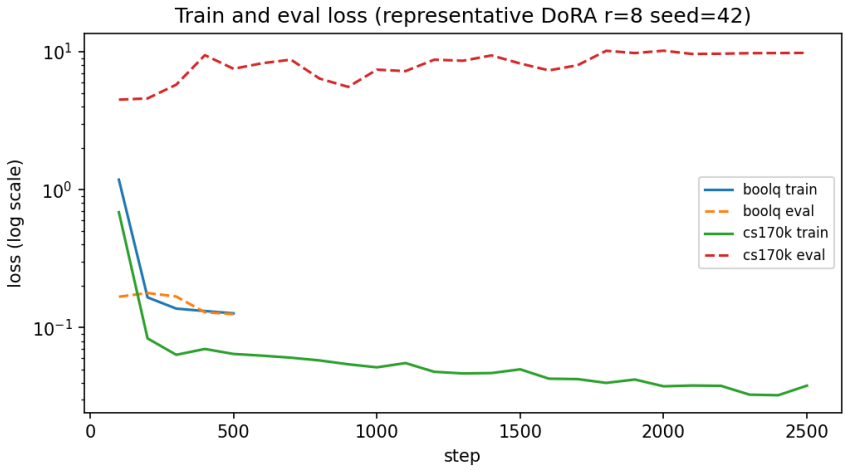

Figure 2: *Train/eval loss (log scale) for a representative run. BoolQ losses settle within a few hundred steps; the cs170k* eval *loss (red) rises toward ∼ 10 even as cs170k* train *loss falls — the first sign of the format mismatch discussed below.*

/ 0.78 DoRA) — an **inverse-rank trend**, opposite to "more capacity is better." It is only a few points wide here and within seed scatter at $n = 3$ — it firms up at the larger budget below. The reading: cs170k specializes the model *away* from pure BoolQ; at low rank that drift is mild and the model keeps its BoolQ ability, while at high rank the adapter commits harder to the broader mixture at BoolQ's expense. DoRA shows no accuracy advantage here either — within seed std at r=4 and r=8, and at r=16 DoRA is in fact slightly *behind* (−0.03, the one direction-consistent gap across all three seeds, still $n = 3$).

The strict-vs-broad parser split (Methods) turns this into a clean story (Figure 3). Broad genmatch (task accuracy) stays flat near 0.82 across rank, but strict genmatch — which counts an answer only if phrased as *true/false* — climbs steeply, from ∼ 0.10 at r=4 to ∼ 0.57 (LoRA) / ∼ 0.63 (DoRA) at r=16. The model answers correctly at every rank (flat broad line); only at higher rank does it adopt the cs170k *true/false* format (rising strict line). The widening gap *is* the format-adaptation signal. This is also where DoRA shows its only hint of the paper's effect — a +0.056 strict-genmatch edge at r=8 and r=16 — but with seed std of 0.20–0.35 and the per-seed direction split, it is a hint, not significance.

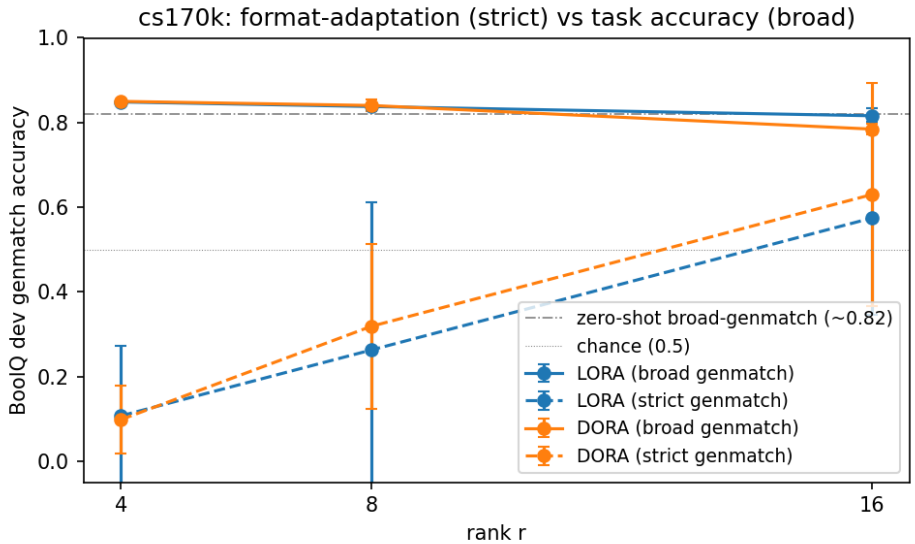

Figure 3: *cs170k, by rank: broad genmatch (solid, task accuracy) stays flat near the 0.82 baseline, while strict genmatch (dashed, format-adaptation strength) climbs with rank — higher-rank adapters lock in the trained true/false format.*

4.3 Scaling the budget (Tier 3): the trends sharpen, and DoRA's real difference appears

Quadrupling the cs170k budget to 10k steps (four fresh seeds) sharpens both findings. The inverse-rank trend widens from ∼ 3–7 percentage points (pp; 3.3 pp LoRA, 6.6 pp DoRA) to ∼ 14 pp (LoRA) / ∼ 16 pp (DoRA) within-method (Figure 4): r=4 still sits at or above baseline (0.84/0.83) while r=16 drops to 0.70/0.66. More training simply lets higher-rank adapters commit harder to the multi-task distribution and sacrifice more BoolQ-specific accuracy — a roughly 2–4× amplification of the Tier-2 signal (≈ 4× for LoRA, ≈ 2.5× for DoRA).

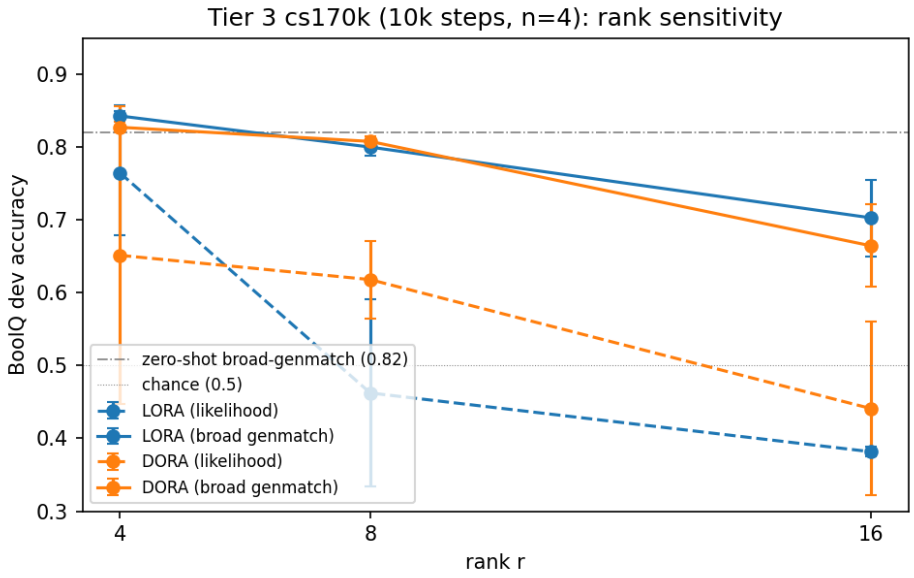

Figure 4: *Tier-3 cs170k (10k steps, $n = 4$). Broad genmatch (solid) shows the sharpened inverse-rank trend; likelihood (dashed) collapses much further — see text.*

At this budget the two-views distinction becomes essential, not optional. Read by the yes/no *likelihood* probe alone, Tier 3 looks catastrophic — the mid- and high-rank cells fall by 0.20–0.40 versus Tier 2 (e.g. LoRA r=8: 0.81 → 0.46). But broad genmatch shows only mild task-accuracy loss (≤ 0.12). The model is not broken; it has *fully* switched to true/false, so the yes/no probe misreads a correct answer as wrong. Figure 5 makes this concrete: training loss converges to ∼ 0.03 while the BoolQ-yes/no monitor loss climbs past 10. **At Tier 3, genmatch is the task-accuracy metric and likelihood is a format-preservation probe.**

Read that way, DoRA's one robust difference from LoRA finally appears — and it is *not* accuracy. At r=8, DoRA's likelihood is 0.62 versus LoRA's 0.46, a +0.16 edge outside DoRA's seed std (∼ 0.05), with three of four paired seeds favouring DoRA — while the matching genmatch gap is ∼ 0. DoRA and LoRA reach the *same* answers, but DoRA's magnitude–direction decomposition resists the cs170k format takeover where LoRA capitulates. Figure 6 shows this directly as the genmatch−likelihood gap: LoRA's jumps to ∼ 0.34 at r=8 (format collapse), DoRA's stays near 0.19. It is tempting to read this as the sharper, $n = 4$ version of the Tier-2 strict-parser hint, but the two point in *opposite* directions. At Tier 2, DoRA leaned slightly *toward* the trained true/false format (more adaptation, within noise); at Tier 3 it instead *held onto* the original yes/no format (more preservation, outside seed std). What is consistent — and all we claim — is that **DoRA's only measurable edge over LoRA is on a format-related axis, not task accuracy**; we do not posit a single mechanism across the two budgets, and the Tier-3 likelihood edge is the statistically cleaner of the two. This is a dimension the project's stated BoolQ-accuracy metric was never designed to surface.

4.4 Cost

DoRA's overhead is the most robust quantitative result of the study: ∼ 3.3× wall-time and ∼ 1.8× peak memory versus LoRA, stable across every rank, regime, and budget (at 10k steps, ∼ 5.3 h vs ∼ 1.6 h per run; ∼ 60 GB vs ∼ 34 GB peak). Tellingly, this is *not* a parameter-count cost: DoRA adds only the magnitude vector — +6.6% trainable parameters at r=8 (22.35 M vs LoRA's 20.97 M) — so the overhead comes from the per-step column-

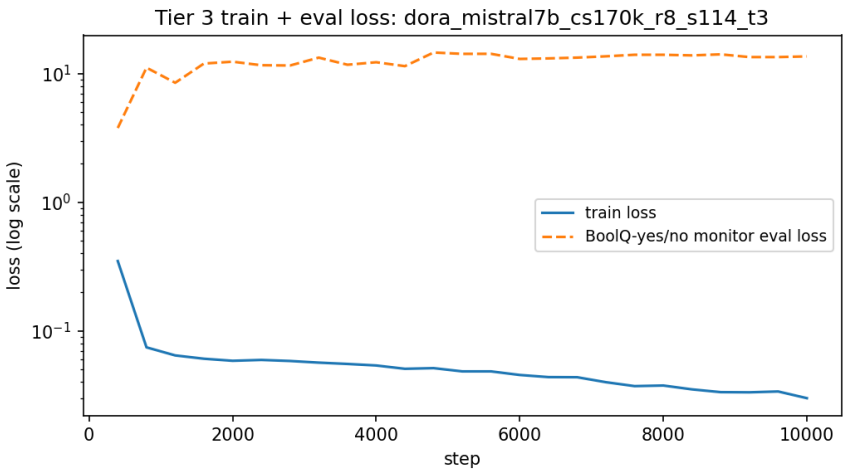

Figure 5: *A representative 10k-step run: train loss falls to ∼ 0.03 while the BoolQ-yes/no monitor eval loss climbs past 10 — the model has fully committed to the trained format, which is why the likelihood probe (but not genmatch) collapses.*

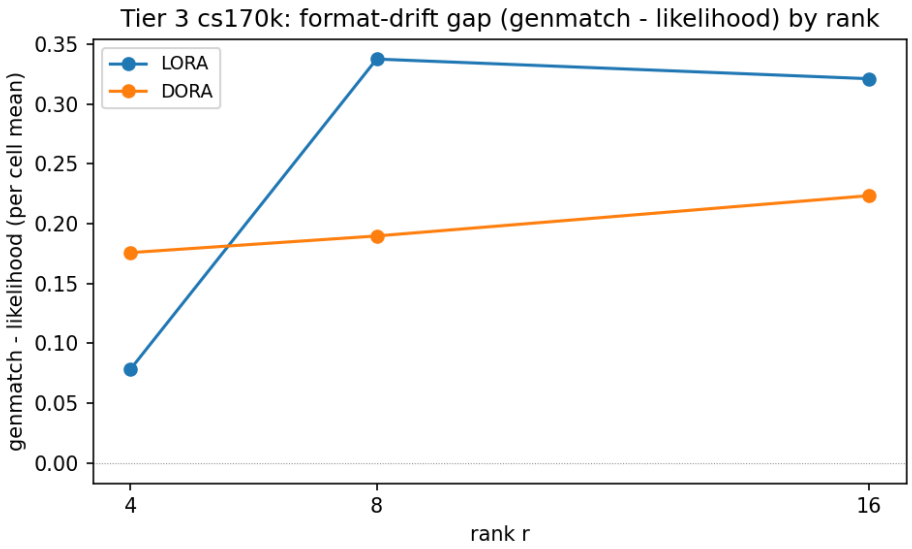

Figure 6: *Tier-3 format-drift gap (genmatch − likelihood) per cell. A larger gap means the likelihood probe has collapsed (format takeover). LoRA's gap spikes at r=8; DoRA's stays lower — DoRA preserves the yes/no format better at mid rank.*

wise renormalization the decomposition requires, not from a larger adapter. Whatever DoRA buys here — format robustness at mid rank, not accuracy — it buys at roughly triple the training cost.

5 Conclusion

We set out to test DoRA's central claim — that its magnitude–direction decomposition beats LoRA, most of all at low rank — by transferring it from the paper's raw-LLaMA setting to an instruction-tuned Mistral-7B, across a per-task and a multi-task regime, two training budgets, and a controlled rank sweep. The claim **does not replicate as a task-accuracy advantage**: at no rank, regime, or budget does DoRA beat LoRA on BoolQ accuracy beyond seed noise. What the decomposition does change is **format robustness** — at mid rank (r=8, 10k steps) DoRA resists the cs170k format takeover better than LoRA (+0.16 on the likelihood probe with the genmatch gap at zero), a difference only visible once task accuracy and format adaptation are separated (a weaker, opposite-pointing Tier-2 hint stayed within noise, so we claim a format-related effect, not a single mechanism). Along the way the multi-task regime revealed an **inverse-rank generalization trend** — low-rank adapters stay at or above the zero-shot baseline while high-rank adapters drift below it — that sharpens by roughly 2–4× with training budget. All of this

comes at DoRA's reconfirmed $\sim 3.3\times$ time and $\sim 1.8\times$ memory cost.

Limitations. The strongest caveats bear directly on the headline null:

- **Statistical power.** Three and four seeds are too few for significance, and the format metrics carry large seed variance (0.16–0.35); the DoRA strict-genmatch edge stays within that noise. Our claims are therefore "no advantage *observed*," not "no advantage *exists*."
- **Training budget.** At 2,500 and 10,000 steps we reach 7.8% and 31% of the paper's $\sim$ 32k-step budget. Training loss converges early, but the format dynamics are still evolving at 10k; a paper-faithful budget could plausibly push the mid-rank DoRA edge to significance.
- **Base-model mismatch.** This is the most likely reason our accuracy result diverges from the paper. An instruction-tuned model is already format-stable and competent at BoolQ, suppressing exactly the format-collapse failure mode the paper's low-rank DoRA advantage is most sensitive to — so LoRA simply has less room to fail here.
- **Metric coupling.** The yes/no likelihood probe stops measuring task accuracy once the model adopts true/false, and the strict-parser numbers are a one-way snapshot. We mitigated both with the broad-parser genmatch, but a parser-independent multi-task metric would be cleaner.
- **Hardware reach.** DoRA's $\sim$ 60 GB peak confines it to 80 GB-class data-centre GPUs, limiting who can use it at all.

Extensions. With more time, the natural next steps are: (i) a paper-faithful $\sim$ 32k-step budget with $n \geq 5$ seeds, to settle whether the format-preservation edge — and any low-rank accuracy gain — crosses significance; (ii) repeating the sweep on the paper's raw LLaMA base, to directly test the "instruction-tuned base suppresses the effect" hypothesis that best explains our null; (iii) evaluating on the *full* cs170k task suite with per-task metrics, so "accuracy" reflects the multi-task objective the adapter actually optimizes rather than BoolQ alone; and (iv) inspecting the DoRA-specific magnitude vector m directly, to understand the mechanism behind its mid-rank format robustness. The broader lesson is methodological: when a method's claimed benefit is measured by generation exact-match, separating *task accuracy* from *format adaptation* is not optional — it is what turned a confusing set of numbers into a coherent, and honest, result.

6 Foundations of the LLM Pipeline (Parts 1–5)

Parts 1–5 of the project notebook build the toolkit the mini-project relies on: using models through the high-level `pipeline`, then rebuilding that pipeline by hand (tokenizer, model, generation), then making it efficient through quantization and parameter-efficient fine-tuning. This section documents our answers to the notebook's discussion questions; long raw transcripts are collected in the Appendix.

6.1 Part 1 — The pipeline

Warnings, and what GPT-2 is (Example 1a). Calling `pipeline("text-generation")` on GPT-2 raises three fixable warnings: no model was supplied (fix: pass `model="gpt2"`); `pad_token_id` defaulted to `eos_token_id` (GPT-2 has no pad token, so HuggingFace reuses the end-of-sequence id 50256 — harmless for single-sequence inference, silenced by passing `pad_token_id` explicitly); and `max_new_tokens` clashing with `max_length` (set only one, preferring `max_new_tokens`, which counts generated tokens only). GPT-2 (Radford et al., 2019) is a 124M-parameter decoder-only transformer trained autoregressively on WebText — a pure generator with no classification head or instruction tuning. Its output stays locally fluent but drifts off-topic and repeats; re-running gives different text because the default sampling is stochastic (`do_sample=True`).

BERT vs GPT-2, and DistilBERT (Example 1b). The default sentiment model is `distilbert-base-uncased-finetuned-sst-2-english`. BERT (Devlin et al., 2019) is an *encoder-only* transformer with bidirectional attention, pretrained on masked-language-modelling and next-sentence prediction — built for *understanding* (classification, QA), whereas GPT-2 is a *decoder* built for generation. DistilBERT (Sanh et al., 2019) is a distilled BERT: $\sim$ 40% fewer parameters ($\sim$ 66M vs 110M), $\sim$ 60% faster, retaining $\sim$ 97% of BERT's GLUE score. The checkpoint name decodes as DistilBERT family, base size, uncased input, fine-tuned on SST-2 (binary movie-review sentiment; Socher et al., 2013). The pipeline returns a `label` (POSITIVE/NEGATIVE) and a softmax `score`; on sarcastic or double-negative inputs the score collapses toward 0.5 or flips outright, showing the model leans on surface keywords.

Two more pipeline tasks (exercise). Beyond generation and sentiment we ran *object detection* with DETR (`facebook/detr-resnet-50`; Carion et al., 2020), returning labelled bounding boxes (example output in the Appendix) and *zero-shot classification* (`valhalla/distilbart-mnli-12-3`, which scores arbitrary candidate labels by treating classification as natural-language inference). Both behaved as expected with no task-specific training.

Why the first `pipeline()` call is slow. The first call for a new model hides several heavy steps behind one line: downloading the config/weights/tokenizer files, instantiating the architecture, loading weights into (GPU) memory, building the tokenizer, and warming up CUDA kernels on the first forward pass. A second call in the same session skips the download but still re-instantiates and re-loads the weights. Once the object exists, inference on a string is just a fast forward pass — the lesson Part 2 builds on: pay the load cost *once* and reuse the components.

6.2 Part 2 — Behind the pipeline

Tokenization (2.1). A token can be a whole word or a sub-word piece. Long or rare strings split into morpheme-like pieces — `tokenization` → `token`+`ization`, `unbelievable` → `un`+`believ`+`able`, and the course code `ELEN90088` fragments into letters and digits — while frequent words (`the`, `Melbourne`) stay single tokens. Sub-word tokenization beats one-integer-per-word for two reasons: it avoids out-of-vocabulary blow-up (any new word is expressible from known pieces, so the model never hits `<unk>`), and it keeps the embedding table small — a 500k-word vocabulary at $d_{\text{model}} = 3072$ would be a $\sim$ 1.5-billion-parameter input layer, versus under 100M for a $\sim$ 32k sub-word vocabulary. BPE, WordPiece and SentencePiece all hit this sweet spot.

Model loading and architecture (2.2). A model's device is a property of its parameters — checked via `model.device`, `next(model.parameters()).device`, or, for split placement under `device_map="auto"`, `model.hf_device_map`. Reading the loaded config (rather than trusting documentation) confirms the architecture: Mistral-7B-Instruct-v0.3 is a 7.25B-parameter decoder with hidden size 4096, 32 layers, 32 query heads but only 8 key/value heads (grouped-query attention, GQA), and no sliding window; Phi-3.5-mini (3.8B; Abdin et al., 2024) instead uses 32/32 heads (plain multi-head attention) and fuses Q/K/V into a single `qkv_proj`. That separate-vs-fused difference matters in Part 5, where LoRA must target the right module names.

6.3 Part 3 — Inference engineering

Generation controls, and a prompt-sense trap (3a). Driving `model.generate()` by hand exposes three knobs: `max_new_tokens` (a cap on generated tokens only), `temperature` (scales logits before softmax — low collapses toward greedy, high flattens and diversifies), and `do_sample` (greedy vs sampling; with `do_sample=False`, temperature is ignored). With sampling on, re-running the same prompt yields a different completion each time. A striking result: asked to "outline an introduction to LLM," Mistral-7B-Instruct misread "LLM" in three of four runs (as *Master of Laws* in two and *Language Modeling Master's Program* in a third) — and lowering the temperature only locked in the *wrong* sense more confidently, since greedy decoding just follows the strongest (here mistaken) prior. Phi-3.5 read it as *Large Language Models* in all five runs. The reliable fix is to disambiguate the prompt (write out "Large Language Models") and then lower the temperature; a stronger base-model prior can also resolve it. Full transcripts are in the Appendix.

Zero-shot vs few-shot sentiment (3b). Re-implementing sentiment as prompted generation on ten deliberately tricky reviews (sarcasm, double negatives, backhanded compliments): the model mostly obeyed the "one word" instruction but broke the *label set*, hedging to "Neutral" on two genuinely ambiguous reviews despite being asked for Positive/Negative. Few-shot helped where the examples teach the task framing — "I can't believe how much I didn't hate this" flips from a wrong "Negative" (zero-shot, reading surface negatives) to a correct "Positive" (few-shot). The overall lift was real but small, 6/10 → 7/10. Few-shot also costs: the four-example prompt is $\sim 3\times$ longer ($\sim$ 220 vs $\sim$ 70 tokens), and since prefill cost and KV-cache grow with prompt length, zero-shot is preferable for high-throughput tasks when its accuracy is acceptable — escalating to few-shot, then to fine-tuning (Part 5), only as needed.

6.4 Part 4 — Compression (quantization)

Why a 16-bit compute dtype for 4-bit weights (4.1). `load_in_4bit` controls *storage*, not *computation*: there is no native 4-bit matmul, so at every forward pass bitsandbytes (Dettmers, 2022) dequantizes each NF4 block back to `bnb_4bit_compute_dtype` (bf16), multiplies it against the 16-bit activations, and discards the temporary. Sixteen

bits is the minimum because the activations and KV cache are themselves 16-bit and the long MLP dot-products (over 14,336 terms) need the accumulation headroom. bf16 specifically — not fp16 — matches Mistral's training dtype, and its wider exponent range avoids overflow on attention logits.

The memory paradox (4.2). Four-bit storage suggests $\sim$ 3.5 GB of VRAM, but the measured 4-bit footprint was 3.75 GiB (against 13.50 GiB for bf16). The gap is fully accounted for: bitsandbytes leaves `embed_tokens` and `lm_head` in bf16 ($\sim$ 0.5 GiB) and adds NF4 double-quant scale metadata ($\sim$ 0.1 GiB), giving an honest floor of $\sim$ 3.85 GiB — within 0.1 GiB of the measurement. (A second surprise — the 4-bit model reporting 3.76 B parameters, about half of nominal — is just bitsandbytes packing two 4-bit weights per byte.) On disk, 13.50 vs 3.85 GiB is a $\sim$ 3.5× compression, short of the naive 4× precisely because of the un-quantized embedding and head.

Perplexity cost (4.3.2). On one short sentence the 4-bit model scored a *lower* perplexity than 16-bit (2.38 vs 2.42, -1.7%) — not a contradiction but variance: quantization perturbs weights like noise, which on a single short sequence can fall either way. Averaged over four sentences the quantization "tax" was $+4.28\%$, within the 3–5% band the QLoRA paper (Dettmers et al., 2023) reports for NF4 on 7B models. Perplexity is preserved tightly under NF4 — but it is a probability metric, not a quality one.

Generation quality (4.3.3). Greedy A/B prompts (sarcasm, a train word-problem, a factual-recall question, a code task) showed quantization had essentially no measurable effect: three of four answers were byte-comparable, and where they differed (the word-problem's final rounding) the 4-bit model was if anything more precise. Both precisions made the *same* factual error — dating *The Master and Margarita* to "the 1940s" — a shared prior in the weights, not a quantization artefact. Full A/B transcripts are in the Appendix. Practically, 4-bit is indistinguishable from 16-bit for interactive use here while saving $\sim$ 10 GiB; the risks lie in long high-temperature generation and rare-token recall, not in this regime.

6.5 Part 5 — Parameter-efficient fine-tuning (PEFT)

Pre-training vs fine-tuning (5.0). Pre-training learns *language* from trillions of unlabelled tokens over months on thousands of GPUs; fine-tuning adapts that foundation to a *task* from far less data in hours. Full fine-tuning still updates every weight — for Mistral-7B, Adam's two optimizer moments alone would need $\sim$ 58 GB — which is why PEFT, updating under 1% of weights, exists. In one line: pre-training gives a model that knows language; fine-tuning gives one that knows your task.

The LoRA hypothesis (5.1). LoRA (E. J. Hu et al., 2021) freezes W_0 and learns a low-rank update, $W = W_0 + \frac{\alpha}{r}BA$ with $r \ll \min(d, k)$, motivated by the finding that a task's weight update has low *intrinsic rank* (Aghajanyan et al., 2020) — the LoRA paper matches full fine-tuning with r as small as 1–8. Because $\Delta W = BA$ has the same shape as W_0, the adapter can be folded back into the weights after training, so there is no inference-time latency penalty. A back-of-envelope for r=16 on the attention projections predicts $\sim$ 6.8M trainable parameters, $\sim$ 0.09% of the model.

Standard LoRA, verified (5.2). Wrapping the 16-bit Mistral with LoRA on `q_proj`/`v_proj` at r=16 reported exactly **6,815,744 trainable parameters (0.094%)**, matching the prediction, with adapter shapes $A = (16, 4096)$ and — under GQA — $B = (1024, 16)$ on `v_proj`. One caveat carried into the mini-project: on Phi-3.5 this same config matches *zero* modules, because Phi fuses Q/K/V into `qkv_proj`; target-module names are model-specific.

QLoRA training (5.3). Attaching the same 0.094%-of-parameters adapter (the 6.82M weights of §5.2 — about 1 in 1,000 of the 7.25 B) to the 4-bit base and training on 100 IMDB reviews, training loss fell from 2.76 to 2.02 over 100 steps, and the incremental VRAM cost of training was only $\sim$ 0.5 GiB — the headline 34 GiB peak was leftover residency from earlier sections, not the adapter — so QLoRA fits comfortably on an 8 GB GPU when only the 4-bit model is resident. The saved adapter was **29.5 MiB,** $\sim$ 485× **smaller** than the base. Evaluation showed the trade-off sharply: in-domain (IMDB) perplexity dropped 59% while out-of-domain (Wikipedia) perplexity rose 54% — classic catastrophic forgetting from over-training on a narrow set. Sentiment accuracy rose from 52% to 86%, but the real change was *format adherence*: the base model produced a parseable label only 54% of the time, versus 100% after tuning. The bottleneck the adapter fixed was format, not understanding — a direct foreshadowing of the mini-project's task-versus-format theme.

Acknowledgements

Training and evaluation ran on the University of Melbourne Research Computing Services *Spartan* HPC (`gpu-h100` partition), on PyTorch 2.6.0 (CUDA 12.4) with the Hugging Face `transformers`, `peft`, `datasets`, and `accelerate` stack.

AI-use disclosure. Generative-AI tools — principally Claude — assisted with explaining LLM concepts for the Parts 1–5 discussion questions, diagnosing the mini-project's code and results, auditing and LaTeX-formatting this report. All experimental design, training runs, results, and conclusions are the author's own; every AI-assisted output was reviewed and verified by the author, who takes full responsibility for the content. No AI tool generated or altered any experimental data or result. External sources consulted are credited in the References.

References

Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., Awan, A. A., Bach, N., et al. (2024). *Phi-3 technical report: A highly capable language model locally on your phone*. https://doi.org/10.48550/arXiv.2404.14219

Aghajanyan, A., Gupta, S., & Zettlemoyer, L. (2020). *Intrinsic dimensionality explains the effectiveness of language model fine-tuning*. https://doi.org/10.48550/arXiv.2012.13255

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. *Computer Vision – ECCV 2020*. https://doi.org/10.1007/978-3-030-58452-8_13

Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., & Toutanova, K. (2019). BoolQ: Exploring the surprising difficulty of natural yes/no questions. *Proceedings of NAACL-HLT 2019*. https://doi.org/10.18653/v1/N19-1300

Dettmers, T. (2022). *Bitsandbytes: 8-bit and 4-bit quantization for PyTorch*. https://github.com/bitsandbytes-foundation/bitsandbytes

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems (NeurIPS)*. https://doi.org/10.48550/arXiv.2305.14314

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*. https://doi.org/10.18653/v1/N19-1423

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. https://doi.org/10.48550/arXiv.2106.09685

Hu, Z., Wang, L., Lan, Y., Xu, W., Lim, E.-P., Bing, L., Xu, X., Poria, S., & Lee, R. K.-W. (2023). LLM-Adapters: An adapter family for parameter-efficient fine-tuning of large language models. *Proceedings of EMNLP 2023*. https://doi.org/10.18653/v1/2023.emnlp-main.319

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., et al. (2023). *Mistral 7B*. https://doi.org/10.48550/arXiv.2310.06825

Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. (2024). DoRA: Weight-decomposed low-rank adaptation. *Proceedings of the 41st International Conference on Machine Learning (ICML)*. https://doi.org/10.48550/arXiv.2402.09353

Mangrulkar, S., Paul, S., Gugger, S., Debut, L., Belkada, Y., & Bossan, B. (2022). *PEFT: State-of-the-art parameter-efficient fine-tuning methods*. https://github.com/huggingface/peft

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language models are unsupervised multitask learners*. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter*. https://doi.org/10.48550/arXiv.1910.01108

Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A. Y., & Potts, C. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. *Proceedings of EMNLP 2013*. https://doi.org/10.18653/v1/D13-1170

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of EMNLP 2020: System Demonstrations*. https://doi.org/10.18653/v1/2020.emnlp-demos.6

A Mini-project — full per-cell results (Tier 2 & Tier 3)

Supporting material relocated from the main body and the Foundations section; none of it counts toward the page limit. Where the raw logs are long, the complete file is linked and a representative excerpt is shown inline. File paths are relative to the report's directory (`Project-LLM/Report/`) in the project's public repository, https://github.com/Remilia-Kirisame/ELEN90088-SOML.

Zero-shot baseline (Mistral-7B-Instruct-v0.3, no adapter): likelihood 0.8235, broad-genmatch 0.8202. All cells below are evaluated on the full BoolQ dev set (3,270 examples).

Table 2: *Tier 2 — per cell (mean ± std over seeds* {42, 1, 2}*; BoolQ 500 steps, cs170k 2,500 steps).*

Train	Method	r	Likelihood	Genmatch (broad)	Genmatch (strict)	Runtime (s)	Peak mem (GB)
BoolQ	DoRA	4	0.8742 ± 0.0067	0.8927 ± 0.0011	—	1189	59.4
BoolQ	DoRA	8	0.8797 ± 0.0079	0.8967 ± 0.0026	—	1181	59.6
BoolQ	DoRA	16	0.8811 ± 0.0081	0.8991 ± 0.0008	—	1181	59.9
BoolQ	LoRA	4	0.8780 ± 0.0040	0.8939 ± 0.0021	—	361	33.3
BoolQ	LoRA	8	0.8803 ± 0.0106	0.8987 ± 0.0036	—	361	33.5
BoolQ	LoRA	16	0.8792 ± 0.0043	0.8996 ± 0.0026	—	356	33.9
cs170k	DoRA	4	0.7499 ± 0.0851	0.8499 ± 0.0055	0.0980 ± 0.0792	5474	59.4
cs170k	DoRA	8	0.8140 ± 0.0244	0.8403 ± 0.0144	0.3183 ± 0.1952	5429	59.6
cs170k	DoRA	16	0.7294 ± 0.0903	0.7844 ± 0.0135	0.6296 ± 0.2627	5426	59.9
cs170k	LoRA	4	0.7788 ± 0.0723	0.8485 ± 0.0025	0.1069 ± 0.1657	1653	33.3
cs170k	LoRA	8	0.8103 ± 0.0131	0.8377 ± 0.0009	0.2624 ± 0.3501	1611	33.5
cs170k	LoRA	16	0.7835 ± 0.0131	0.8159 ± 0.0189	0.5742 ± 0.2282	1603	33.9

Table 3: *Tier 2 — DoRA − LoRA gaps (positive = DoRA higher).*

Regime · metric	r=4	r=8	r=16
BoolQ · likelihood	−0.0038	−0.0006	+0.0019
BoolQ · genmatch (broad)	−0.0012	−0.0019	−0.0005
cs170k · likelihood	−0.0288	+0.0037	−0.0541
cs170k · genmatch (broad)	+0.0014	+0.0025	−0.0315
cs170k · genmatch (strict)	−0.0089	+0.0559	+0.0555

Table 4: *Tier 3 — per cell (mean ± std over seeds* {114, 514, 1919, 810}*; cs170k, 10,000 steps).*

Train	Method	r	Likelihood	Genmatch (broad)	Runtime (s)	Peak mem (GB)
cs170k	DoRA	4	0.6514 ± 0.2044	0.8272 ± 0.0071	19324	59.4
cs170k	DoRA	8	0.6180 ± 0.0530	0.8078 ± 0.0075	19134	59.6
cs170k	DoRA	16	0.4411 ± 0.1191	0.6645 ± 0.0565	19110	59.9
cs170k	LoRA	4	0.7644 ± 0.0856	0.8427 ± 0.0148	5957	33.3
cs170k	LoRA	8	0.4624 ± 0.1280	0.8000 ± 0.0114	5906	33.5
cs170k	LoRA	16	0.3817 ± 0.0069	0.7029 ± 0.0526	5880	33.9

Tier 3 — DoRA − LoRA gaps: likelihood +0.1557 at r=8 (−0.1131 at r=4, +0.0593 at r=16); broad-genmatch within ±0.04 at every rank (−0.0155 / +0.0078 / −0.0384).

Cross-tier (Tier 3 − Tier 2) deltas, cs170k: likelihood collapses (LoRA −0.014 / −0.348 / −0.402 at r=4/8/16; DoRA −0.099 / −0.196 / −0.288), while broad genmatch holds far better (LoRA −0.006 / −0.038 / −0.113; DoRA −0.023 / −0.033 / −0.120) — the format takeover, not a task collapse.

B Mini-project — representative raw generations (format adaptation)

Inspecting 20 generations per run with `evaluate.py --debug-print` resolved the parser puzzle:

Table 5: *Representative generations: surface form (yes/no vs. true/false) at low vs. high rank.*

Run	*r*	Model emits	Strict	Broad
`lora_r4_s1`	4	"the correct answer is **yes/no**"	0.0003	0.846
`dora_r16_s1`	16	"the correct answer is **true/false**"	0.793	0.799

Low-rank adapters answer in the prompt's *yes/no* (so the strict true/false parser scores ≈ 0 despite correct answers); high-rank adapters override the prompt with the trained *true/false*. Both are answering correctly — the difference is surface form, which is exactly what the broad-vs-strict split separates.

C Part 3b — full sentiment predictions (ten tricky reviews)

Greedy decoding; raw CSV at `../Results-ipynb/sentiment_results.csv`.

Table 6: *Part 3b — full sentiment predictions on ten tricky reviews (greedy decoding).*

#	Review (abridged)	Expected	Zero-shot	Few-shot	Note
1	"…worried it would be terrible, but it actually wasn't bad…"	Positive	Positive	Positive	both correct
2	"Oh great, another 'groundbreaking' superhero movie…"	Negative	Negative	Negative	sarcasm caught
3	"The plot was non-existent…yet I couldn't look away."	Positive	Negative	Negative	both wrong (surface negativity)
4	"…not that the food was cold…wasn't particularly warm either."	Negative	Neutral	Neutral	instruction violated — hedged
5	"If you enjoy watching paint dry, you'll love this movie!"	Negative	Negative	Negative	sarcasm caught
6	"I've had better, but I've certainly had much, much worse."	Positive	Neutral	Neutral	instruction violated — ambiguous
7	"Everything about this place is perfect, except for the service, the food, and the price."	Negative	Negative	Negative	backhanded
8	"I can't believe how much I didn't hate this."	Positive	Negative	**Positive**	few-shot fixes double negation
9	"This is the best example of a terrible movie I have ever seen."	Negative	Negative	Negative	backhanded
10	"The only thing more disappointing than the ending was paying for the ticket."	Negative	Negative	Negative	both correct

Scoreboard vs the human label: zero-shot 6/10, few-shot 7/10 (rows 4 and 6 hedge to Neutral in both).

D Part 3a — generation transcripts (the "LLM" sense-ambiguity)

Complete five-run logs for both models at `../Results-ipynb/Part-3a.md`. Opening lines, showing the sense each model assigns to "LLM":

Mistral-7B-Instruct (prompt: "Outline an introduction to LLM…"):

- Run 1 (T=0.7): "…Introduction to **Master of Laws (LLM)** Program…" — law degree (wrong sense)
- Run 2 (T=0.7): "…Bridging the Gap between Mathematics, Programming, and Artificial Intelligence" — AI-adjacent, never names *Large Language Models*
- Run 3 (T=0.1): "…Introduction to **Master of Laws (LLM)**…" — greedy, confidently wrong
- Run 4 (T=1.3): "…**Language Modeling Master's Program**…" — creative compromise

- Run 5 (specific prompt "**Large Language Models**", T=0.2): "…Introduction to **Large Language Models**: A Comprehensive Overview…" — correct

Phi-3.5-mini read it as *Large Language Models* in all five runs even with the original ambiguous prompt (e.g. Run 1: "Introduction to **Large Language Models (LLMs)** for Students with Math and Programming Backgrounds").

E Part 4.3.3 — 16-bit vs 4-bit A/B generations (greedy)

Complete transcripts at `../Results-ipynb/Part-4.3.3.md`. Summary of the four probes:

Table 7: *Part 4.3.3 — 16-bit vs. 4-bit greedy A/B generations (summary of four probes).*

Prompt	16-bit	4-bit	Verdict
Sarcasm ("paint dry")	correct, fluent	correct, fluent	both pass, same explanation
Train word-problem	meet at **14:28**	meet at **14:27**	both pass; 4-bit rounds more precisely
The Master and Margarita	"in the **1940s**"	"in the **1940s**"	both **wrong**, identical (shared prior, not quantization)
`is_palindrome`	correct 3-line fn	identical 3-line fn	both pass, byte-identical code

F Part 5.3 — QLoRA training-loss trajectory

100 optimizer steps (= 4 epochs over 100 IMDB reviews), logged every 5 steps:

2.76, 2.70, 2.52, 2.47, 2.63, 2.38, 2.33, 2.34, 2.30, 2.36, 2.13, 2.22, 2.18, 2.03, 2.18, 2.12, 1.93, 1.98, 1.92, 2.02 — final mean training loss **2.27**.

The step-5 `grad_norm = NaN` is normal fp16 `GradScaler` warmup (it detects the first-step overflow, skips that optimizer step, and halves the loss-scale); every `grad_norm` from step 10 on is finite, confirming the scaler stabilized.

G Part 1 — object-detection output

`facebook/detr-resnet-50` on the sample parrots image, with predicted boxes and confidence scores drawn on:

Figure 7: *Object detection (DETR-ResNet-50): two birds localized with bounding boxes and confidence scores.*